Politique de commit

Inception-of-Things - dépôt du groupe pjurdana

1. Format du message

Le dépôt suit la convention **Conventional Commits** :

```
type(scope): description
```

- **type** : la nature du changement (voir tableau 2), en minuscules.
- **scope** : la partie du projet touchée, entre parenthèses. Facultatif si le changement concerne tout le dépôt.
- **description** : ce que fait le commit, en une ligne.

2. Types autorisés

Type	Quand l'utiliser	Exemple du dépôt
`feat`	Nouvelle fonctionnalité	`feat(p1): install K3s agent from the host cache with online fallback`
`fix`	Correction d'un bug ou d'un comportement faux	`fix(p2): fall back to app3 when the host matches neither app1 nor app2`
`refactor`	Réécriture sans changement de comportement	`refactor(p1): rewrite provisioning as server and agent scripts with private default route`
`perf`	Amélioration des performances	`perf(p1): use KVM paravirt provider and 2 CPU / 2 GB per node for nested virtualization`
`docs`	Documentation, commentaires, checklists	`docs: add evaluation walkthrough for p1 and p2`
`chore`	Maintenance : .gitignore, déplacement de fichiers, outillage	`chore: ignore local test scripts`

3. Scopes

Scope	Contenu
`p1`	Dossier `p1/` : Vagrantfile, scripts server et agent
`p2`	Dossier `p2/` : Vagrantfile, script d'installation, manifests `confs/`
`p3`	Dossier `p3/` : K3d, Argo CD
`bonus`	Dossier `bonus/` : Gitlab
`make`	`Makefile`
(aucun)	Fichiers à la racine ou changement transverse

4. Règles de rédaction

- En **anglais**, à l'**impératif** : `add`, `fix`, `use` (pas `added` ni `adds`).
- En **minuscules** après les deux-points, **sans point final**.
- **72 caractères maximum** pour la ligne entière.
- Dire **ce que change** le commit et, si utile, **pourquoi** - pas le détail du code.
- Si une explication est nécessaire, ajouter un corps séparé par une ligne vide.

Mauvais	Bon
`FIX: syntaxe error`	`fix(p3): quote the namespace in setup.sh`
`Save vm crash`	`chore: save work in progress before VM rebuild`
`ADD: whole part 2 working for web apps`	`feat(p2): deploy app1, app2 and app3 behind a Traefik ingress`
`fix stuff.`	`fix(p2): wait for Traefik to answer on port 80 before reporting apps deployed`

5. Un fichier par commit

- **Un `git add` = un fichier**, **un commit = un message** qui décrit ce fichier.
- Jamais `git add .` ni `git add -A` : on choisit ce qui part, fichier par fichier.
- Si plusieurs fichiers forment un seul changement logique, on les commite quand même séparément, chacun avec son message, dans un ordre qui garde le dépôt fonctionnel.

```
git add p2/confs/ingress.yaml
git commit -m "fix(p2): use ingressClassName instead of the deprecated ingress.class
annotation"

git add p2/scripts/install_server.sh
git commit -m "fix(p2): wait for Traefik to answer on port 80 before reporting apps deployed"
```

6. Avant de commiter

1. `git status` : vérifier la liste des fichiers modifiés.
2. `git diff <fichier>` : relire le changement.
3. Pour un `feat`, `fix` ou `perf` : avoir testé (`make p1` / `make p2` / `make p3-setup` et les vérifications de `eval.md`).
4. `git add <fichier>` puis `git diff --staged` : vérifier ce qui part réellement.

7. Ne jamais commiter

Quoi	Pourquoi	Protection
`.vagrant/`	État des VMs, clés SSH, token K3s	`.gitignore`
`*.box`	Images de VM (plus de 1 Go)	`.gitignore`
`.env`, `*.kubeconfig`	Secrets, accès aux clusters	`.gitignore`
`test_*.sh`	Scripts de test locaux, hors rendu	`.gitignore`

`iot-subject.pdf`, `iot-correction.pdf`	Documents de 42, pas notre travail	`.gitignore`

8. Corriger une erreur

Situation	Commande
Fichier ajouté par erreur (pas encore commité)	`git restore --staged <fichier>`
Message du dernier commit faux (pas encore poussé)	`git commit --amend -m "nouveau message"`
Oubli d'un changement dans le dernier commit (pas encore poussé)	`git add <fichier> && git commit --amend --no-edit`
Commit déjà poussé	Ne pas réécrire l'historique : faire un nouveau commit `fix`

9. Branche et push

- On travaille sur `main`.
- On pousse (`git push`) uniquement quand la partie touchée a été testée et fonctionne.
- Avant une soutenance : `git status` doit être propre et tout doit être poussé (le correcteur fait un `git clone`).